Worksheet-6

Name: Aman Kumar
Branch: MCA General
Semester: II
Subject Name: Technical Training

UID: 25MCA20128
Section/Group: 25MCA_KAR-1
Date of Performance: 03-03-26
Subject Code: 25CAP-652

Aim of the Session:

The aim of this practical session is to learn how to create, query, and manage SQL views to simplify database queries and provide a layer of abstraction and security for end-users. Views help present filtered, joined, or summarized data without exposing the complete underlying table structure.

Objective of the Session:

The objectives of this practical session are:

- To understand data abstraction using SQL views.
- To restrict access to sensitive columns through views.
- To simplify complex joins by presenting them as virtual tables.
- To create simple and complex views for reporting purposes.
- To learn syntax for creating, replacing, and dropping views.

Practical / Experiment Steps:

Step 1: Creating a Simple View for Data Filtering

- Implementing a view to display only active employees from the Employee table.

Step 2: Creating a View for Joining Multiple Tables

- Creating a view that joins Employees and Departments tables for simplified data retrieval.

Step 3: Advanced Summarization View

- Creating a view to show department-wise employee count and average salary.

Procedure of the Practical:

1. Start the system and log in to the computer.
2. Open the PostgreSQL client tool (psql / pgAdmin).
3. Create base tables for view demonstration.

```
CREATE TABLE Departments (
   DeptID SERIAL PRIMARY KEY,
   DeptName VARCHAR(50)
);

CREATE TABLE Employees (
   EmpID SERIAL PRIMARY KEY,
   EmpName VARCHAR(50),
   DeptID INT REFERENCES Departments(DeptID),
   Salary DECIMAL(10,2),
   Status VARCHAR(10)
);
```

4. Insert sample records into tables.

```
INSERT INTO Departments (DeptName) VALUES
('HR'),
('IT'),
('Finance'),
('Sales');

INSERT INTO Employees (EmpName, DeptID, Salary, Status) VALUES
('Amit', 2, 60000, 'Active'),
('Riya', 1, 45000, 'Active'),
('Kunal', 2, 70000, 'Inactive'),
('Sneha', 3, 50000, 'Active'),
('Rahul', 4, 40000, 'Active'),
('Anita', 2, 65000, 'Active');
```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

	deptid [PK] integer	deptname character varying (50)
1	1	HR
2	2	IT
3	3	Finance
4	4	Sales

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of 1

	empid [PK] integer	empname character varying (50)	deptid integer	salary numeric (10,2)	status character varying (10)
1	1	Amit	2	60000.00	Active
2	2	Riya	1	45000.00	Active
3	3	Kunal	2	70000.00	Inactive
4	4	Sneha	3	50000.00	Active
5	5	Rahul	4	40000.00	Active
6	6	Anita	2	65000.00	Active

5. Creating a Simple View (Active Employees).

```
CREATE OR REPLACE VIEW ActiveEmployees AS
SELECT EmpID, EmpName, DeptID, Salary
FROM Employees
WHERE Status = 'Active';
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	empid integer	empname character varying (50)	deptid integer	salary numeric (10,2)
1	1	Amit	2	60000.00
2	2	Riya	1	45000.00
3	4	Sneha	3	50000.00
4	5	Rahul	4	40000.00
5	6	Anita	2	65000.00

6. Creating a Join View (Employee-Department Details).

```
CREATE OR REPLACE VIEW EmployeeDeptView AS
SELECT e.EmpID, e.EmpName, d.DeptName, e.Salary
FROM Employees e
JOIN Departments d
ON e.DeptID = d.DeptID;
```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of 1

	empid integer	empname character varying (50)	deptname character varying (50)	salary numeric (10,2)
1	1	Amit	IT	60000.00
2	2	Riya	HR	45000.00
3	3	Kunal	IT	70000.00
4	4	Sneha	Finance	50000.00
5	5	Rahul	Sales	40000.00
6	6	Anita	IT	65000.00

7. Creating a Summary View (Department Statistics)

```
CREATE OR REPLACE VIEW DeptSummary AS
SELECT d.DeptName,
    COUNT(e.EmpID) AS TotalEmployees,
    AVG(e.Salary) AS AvgSalary
FROM Departments d
LEFT JOIN Employees e
ON d.DeptID = e.DeptID
GROUP BY d.DeptName;
```

Data Output Messages Notifications

Showing rows: 1 to 4 Page No: 1 of 1

	deptname character varying (50)	totalemployees bigint	avgsalary numeric
1	Finance	1	50000.000000000000
2	Sales	1	40000.000000000000
3	IT	3	65000.000000000000
4	HR	1	45000.000000000000

8. Verify the output after execution.
9. Note down the results obtained.
10. Save the work and take screenshots for record.

I/O Analysis:

Input Provided:

- SQL CREATE VIEW statements.
- Employee and Department sample tables.
- Queries on simple, join, and aggregate views.

Output Generated:

- Filtered list of active employees.
- Combined employee-department data.
- Department-wise employee count and average salary.

Learning Outcomes:

- Understanding abstraction using SQL views.
- Ability to hide complex joins behind virtual tables.
- Knowledge of restricting sensitive data via views.
- Mastery of creating simple and complex views.
- Practical exposure to real-world applications like payroll and reporting systems.